

Latent Perception-to-Reasoning Interfaces for ECG Triage

Phase 1 technical report: from single-event adapter to multi-event reasoning

Author: Alif Tasbir

Date: 28 September 2026

Status: post-viva extension of the MSc dissertation, working towards journal submission

Code and records: project repository (branch phase1-step1); pre-registered analysis plan docs/analysis_plan.md (Deviations 1–17); run log TASKS.md; every number below comes from a committed results file.

1. Executive summary.....	2
2. Starting point: the dissertation	2
3. How Phase 1 was run	3
4. Re-testing the single-event claims.....	3
5. What changed in the system	4
5.1 Perception: RR-timing branch	4
5.2 Multi-event adapter.....	4
5.3 Schema-constrained decoding.....	5
6. Efficiency results	5
7. Accuracy results.....	6
7.1 Primary test (pre-registered, Deviation 16).....	6
7.2 Calibrated comparison (Deviation 17).....	7
7.3 Why text fails with many events	7
7.4 Natural-prevalence windows and false alarms	8
7.5 Against true annotations	8
8. Auditability.....	8
9. Status of the research questions	9
10. Limitations and threats to validity.....	9
11. Proposed next steps and questions for the supervisor	10
Appendix A. Pre-registered deviations	10

1. Executive summary

The dissertation proposed a small learned adapter that passes an ECG model's internal representation directly into a frozen language model (Gemma 4 E4B) as a few virtual tokens, instead of a JSON text interface. Phase 1 re-tested its claims under a pre-registered, multi-seed protocol and then extended the design from one heartbeat per call to many heartbeats per call. The main results:

- **The single-event claims did not hold against a fair baseline.** Against a compact JSON prompt the adapter saves no prompt-processing time (about -2%), and it is less accurate (0.79 vs 0.97 balanced accuracy). The dissertation's 13.3% latency gain came from shorter outputs, not from the interface, and its 95% accuracy was one favourable training run (3 runs: 78.8–95.0%).
- **The advantage appears when many events share one prompt.** A trained multi-event adapter keeps prompt processing flat (about 183 ms from 1 to 50 events), while text grows with every event: -24% / -50% / -78% time to first token at 10 / 20 / 50 events, and -4% / -14% / -36% time to the urgency decision.
- **Accuracy at scale (pre-registered, 1,195 test windows, 22 held-out patients, 3 seeds, patient-level confidence intervals):** the adapter is non-inferior to text at 5 events and more accurate at 10, 20 and 50 events (+0.22, +0.31, +0.23 balanced accuracy) under default decoding.
- **Important qualification:** text's weakness is largely a conservative escalation threshold. When both arms get a threshold tuned on validation data, the adapter is non-inferior (not superior) at 10 and 20 events. The defensible headline is therefore **equal accuracy at 19–78% lower prompt-processing cost**.
- **Auditability holds per event.** Each event's label, tier and urgency flag can be decoded from its slot of virtual tokens as well as from the encoder vector itself, at any of 50 positions. What is lost (heart rate, RR interval, run length) is lost in the encoder, not the adapter.
- **Main open weakness:** the adapter raises false alarms on 2.5–11.5% of routine windows (text: 0%), concentrated on windows containing a low-confidence normal beat. Threshold calibration did not fix this; hard-negative training is the planned remedy.

2. Starting point: the dissertation

The pipeline paired a CNN-LSTM beat classifier (MIT-BIH, inter-patient DS1/DS2 split) with Gemma 4 E4B (4-bit, frozen). Arm A sent each beat's HealthEvent JSON as text; Arm B projected the classifier's 32-d context vector into 4 virtual tokens. The task was the urgency tier (routine / priority / urgent) of one beat, defined by a fixed escalation rule on the classifier's output.

Measure (80-event set)	Arm A (JSON)	Arm B (adapter)
Mean prompt tokens	645.2	486.0 (-24.7%)
Mean generation latency	9,892 ms	8,574 ms (-13.3%)
Task accuracy	100%	95.0% (seeds: 85.0% ± 8.7 pp)
Class recoverability	100% by construction	67.6% (probe)

Table 1. Dissertation headline results (dissertation Table 4.1).

Research questions: RQ1, does the adapter reduce token count, latency and memory; RQ2, does it preserve task accuracy; RQ3, what does a vector interface cost in auditability, and can that cost be measured.

3. How Phase 1 was run

- **Pre-registration.** Every change of design was written into docs/analysis_plan.md as a numbered deviation before any data it affected (17 deviations, each with its reason). Confirmatory and exploratory analyses are labelled as such.
- **Fair baselines.** Besides the dissertation's full JSON (A-full), a compact JSON (A-compact: label, confidence, run length, SQL) and a label-only arm were added; A-compact is the strongest text baseline and is used throughout.
- **Seeds and statistics.** At least 3 training seeds per recipe; paired bootstrap confidence intervals; for the final test, a record-level (patient-cluster) bootstrap, because windows from one patient are correlated.
- **Held-out patients.** Adapters train on DS1 (19 records) with 3 held-out DS1 records for validation; all headline results are on DS2 (22 records never used for training or tuning).
- **Reproducibility.** Resumable, bit-identical training checkpoints (unit-tested); atomic result files; 62 automated tests pass.

4. Re-testing the single-event claims

Dissertation claim	Phase 1 finding	Evidence
95% accuracy	One favourable draw; seeds give 81.3% and 78.8%	E2 seed variance
-24.7% prompt tokens	Against full JSON only; against compact JSON the adapter uses more tokens end to end (486 vs 458); interface tokens 4 vs ~57	Step 4 baseline family
-13.3% latency	An output-length effect: with output length fixed, arms differ by $\leq 3\%$; decode speed identical (~169 ms/token)	Step 5 forced-length timing
Trained with 3 full-batch updates	Erratum: the code made 192 per-example updates (64×3 epochs)	adapter_training.py
67.6% class recoverability	Reproduced (69.6%); the ceiling is the encoder, confirmed	Step 3 probe, item 6 decoder

Table 2. What happened to each single-event claim.

Conclusion: for single events the latent interface has no cost advantage over a well-designed text prompt and is less accurate. The contribution had to move to the setting where the interface can matter: many events in one context.

5. What changed in the system

Dissertation (single event)



Phase 1 (multi-event)



Figure 1. Pipeline before (single event) and after Phase 1 (multi-event). The highlighted block is the component trained in this work.

5.1 Perception: RR-timing branch

A 5-feature RR-interval branch (pre-RR, post-RR, local mean RR and two ratios) was added to the CNN-LSTM's context stage. S-beat sensitivity rose from 8% to 38%, and class recoverability from the 32-d vector from $69.6\% \pm 9.9$ to $84.6\% \pm 5.5$. Cost: one beat of latency (the post-RR interval) and a single training seed so far.

5.2 Multi-event adapter

- N context vectors $\rightarrow N \times 4$ virtual tokens: a per-event linear projection, learned event-position embeddings (up to 50 slots), and per-token L2 normalisation with a learnable scale (this removed the norm drift that broke generation at large N).
- Task: the most urgent tier among N consecutive beats, plus which beat. Deterministic training targets in the same JSON schema.
- Training recipe r3 (final): batch 1, AdamW $5e-4$ with warm-up and cosine decay, validation every quarter epoch with early stopping from 2 epochs, 375 class-balanced windows ($N = 1, 5, 10, 20, 50$). Memory-saving (logits only for target tokens, gradient checkpointing) keeps peak GPU memory at 9.5–10.3 GB on a 12 GB card.
- The first recipe (r1, 8-step gradient accumulation, 111 optimizer steps) under-trained: one seed failed completely (0.38 on its own training windows). Diagnosing and fixing this was pre-registered as Deviation 11.

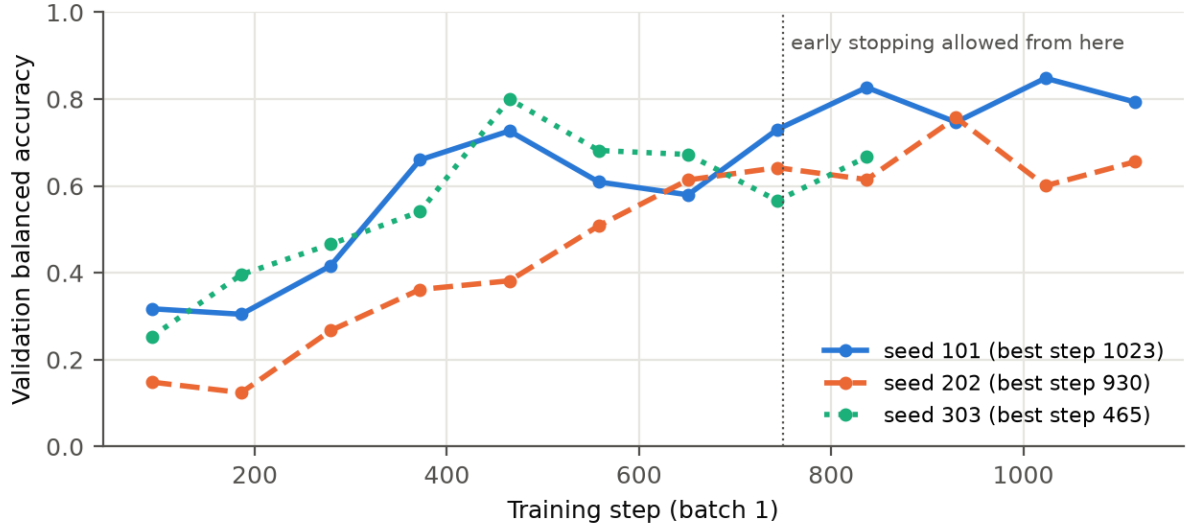


Figure 2. Validation balanced accuracy during training, recipe r3, three seeds. All seeds learn the task; the best checkpoint per seed is selected on validation only.

5.3 Schema-constrained decoding

Free generation produced malformed JSON 9–20% of the time (misnamed fields, loops). Every arm now uses the same constrained decoder: the JSON structure is forced, the tier is restricted to the three valid words and chosen by the model's own scores, and free-text fields cannot break the JSON. Parse rate is 100% for every arm; a check confirmed the decoder changes no tier (380/380 windows identical before and after its last correction).

6. Efficiency results

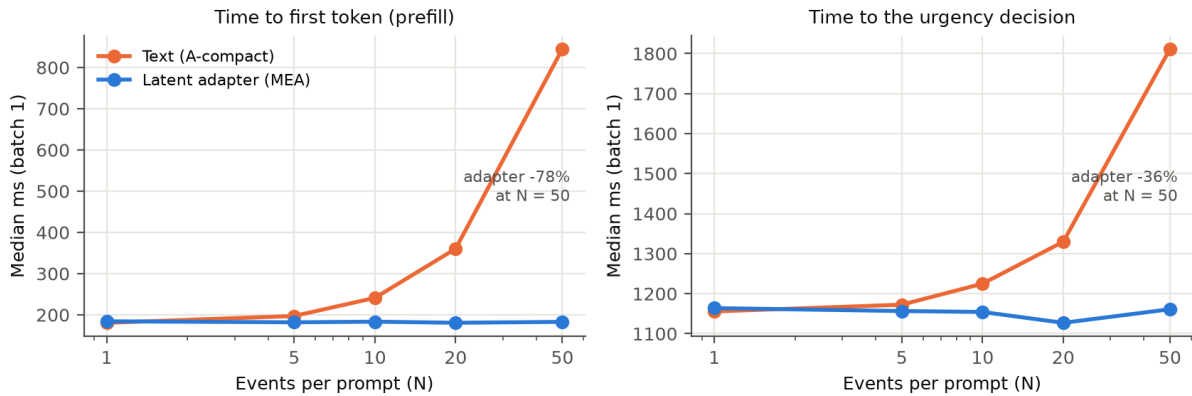


Figure 3. Median latency per request (batch 1, 21 DS2 windows per N). Left: time to first token. Right: time until the urgency tier is generated (token 7 of the constrained answer).

Events per prompt	Prefill vs text	Time to decision vs text	Other
1	0%	+1%	no advantage
5	−8%	−1%	no advantage
10	−24%	−4%	−79 MB context memory
20	−50%	−14%	−177 MB
50	−78% (845 → 183 ms)	−36% (1,812 → 1,161 ms)	−460 MB; energy 391 → 230 J

Table 3. Cost of the latent interface relative to compact text (E3, E3b, Deviation 14). End-to-end response time is dominated by generating the answer (~160 ms/token for every arm), so full-response latency depends on answer length, not on the interface.

7. Accuracy results

7.1 Primary test (pre-registered, Deviation 16)

Test set: 1,195 DS2 windows (22 patients), balanced by tier and by the class of the most urgent beat, including 400 natural-prevalence windows for false alarms. Primary statistic: adapter (3-seed mean) minus A-compact balanced accuracy, with a patient-level bootstrap confidence interval; non-inferiority margin −0.05.

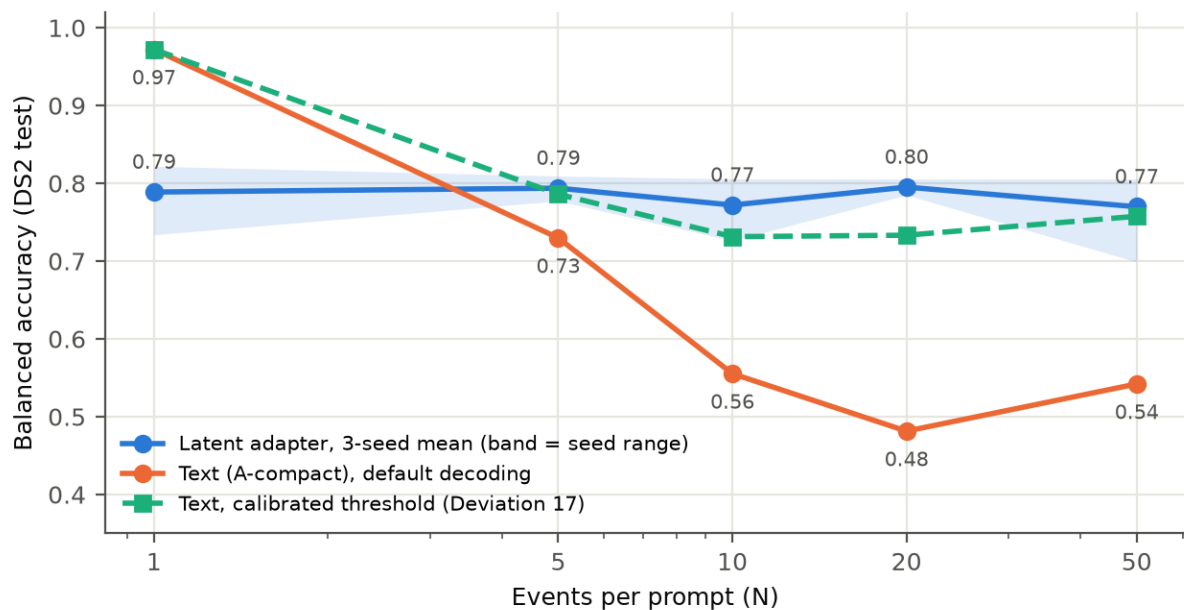


Figure 4. Balanced accuracy on the DS2 test set by window size. The adapter stays flat at about 0.8; default text falls sharply from $N = 5$; text with a calibrated threshold recovers most of the gap.

N	Windows	Text (default)	Adapter (mean)	Difference [patient-level 95% CI]	Verdict
1	177	0.97	0.79	—	text better
5	175	0.73	0.79	+0.06 [−0.02, +0.15]	non-inferior
10	169	0.56	0.77	+0.22 [+0.13, +0.31]	superior
20	157	0.48	0.80	+0.31 [+0.19, +0.43]	superior
50	117	0.54	0.77	+0.23 [+0.07, +0.37]	superior

Table 4. Primary result under default decoding. A 3-seed majority vote lifts the adapter to 0.81–0.83 at every N .

7.2 Calibrated comparison (Deviation 17)

A bias on the routine score at the tier step, chosen on the DS1 validation set only, lifts text from 0.48–0.56 to 0.73–0.76 at $N \geq 10$. With both arms calibrated, the adapter is non-inferior at $N = 10$ and 20 and inconclusive at $N = 5$ and 50. This is the fair comparison and is reported next to the default one.

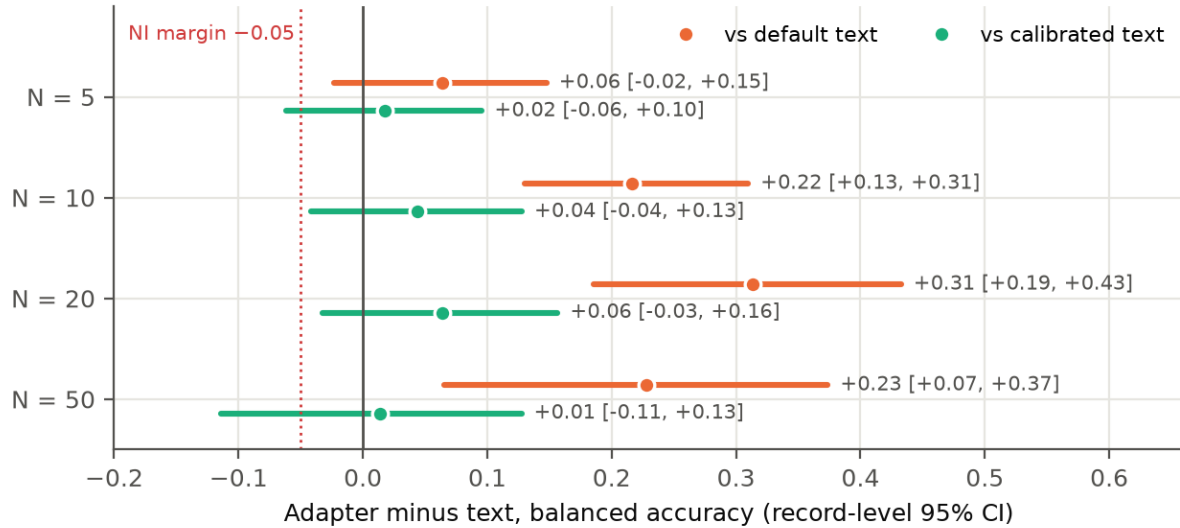


Figure 5. Adapter minus text with patient-level 95% confidence intervals, against default and calibrated text. Dotted red line: non-inferiority margin.

7.3 Why text fails with many events

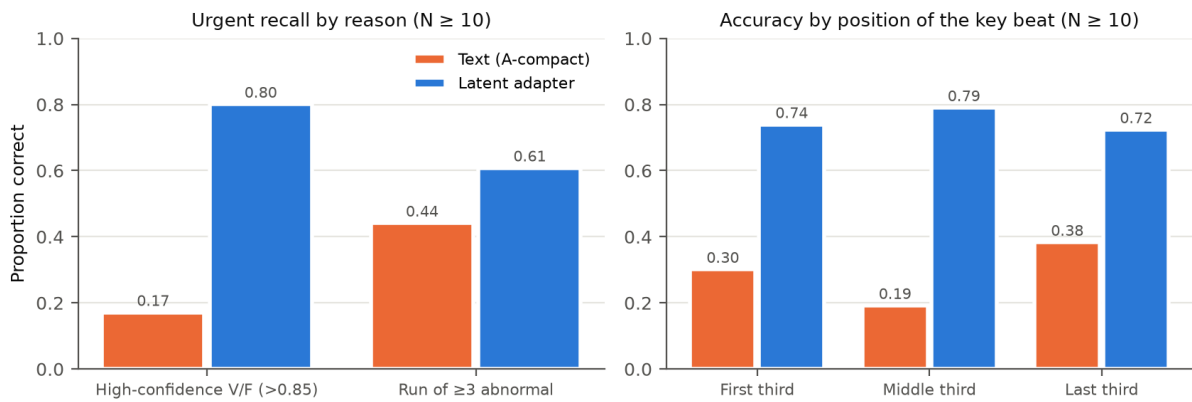


Figure 6. Left: text misses urgent windows caused by a single high-confidence V/F beat (a numeric threshold, 0.85, buried in a long list) far more than runs of abnormal beats. Right: text is worst when the key beat is in the middle of the list, the 'lost in the middle' effect; the adapter shows no position effect.

Both mechanisms are documented in the literature: position effects in long contexts (Liu et al., TACL 2024; Hsieh et al., Findings of ACL 2024) and label bias corrected by calibration (Zhao et al., ICML 2021). A fairness point: the text arm is zero-shot, whereas the adapter was trained on this task; a trained text baseline is planned.

Confusion at N = 50 (row-normalised)

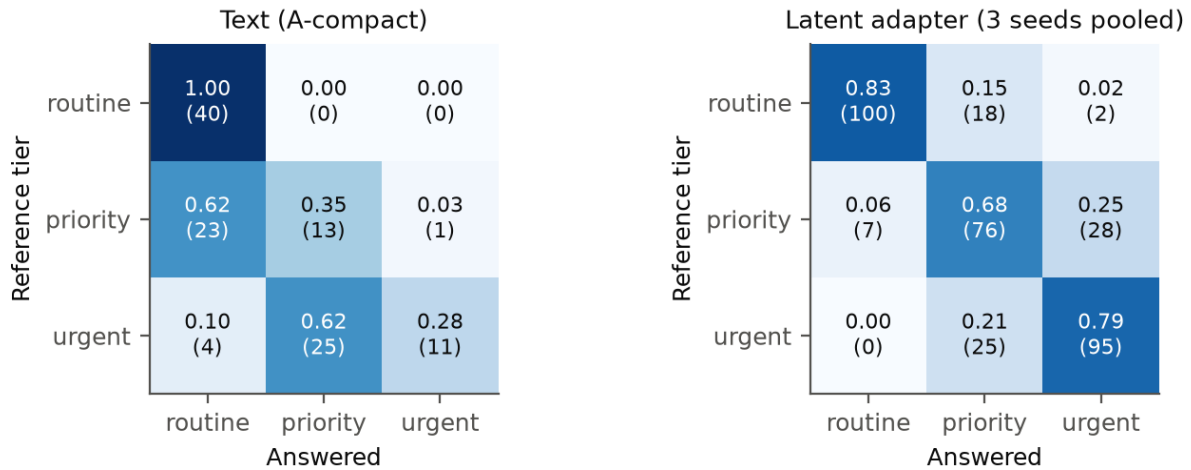


Figure 7. Confusion at N = 50. Text under-escalates (priority → routine, urgent → priority) and never raises a false alarm; the adapter rarely calls an abnormal window routine but sometimes escalates routine windows.

7.4 Natural-prevalence windows and false alarms

N	Accuracy: text	Accuracy: adapter (3 seeds)	False alarms: text	False alarms: adapter
5	0.84	0.88	0%	4.8–8.1%
10	0.61	0.81–0.87	0%	7.7–11.5%
20	0.60	0.88–0.91	0%	8.3–10.4%
50	0.56	0.88–0.90	0%	2.5–10.0%

Table 5. Natural-prevalence windows (100 per N; 40–62 routine windows per N).

The adapter's false alarms sit on windows containing a normal beat classified with low confidence (median lowest confidence 0.56 vs 0.99; an abnormal class as strong second choice). Only 12% of them contain a truly abnormal beat. Threshold calibration could not target them because the validation set has almost no such windows; hard-negative training is proposed.

7.5 Against true annotations

Scoring escalation against the MIT-BIH annotations (does the window contain a truly abnormal beat?) shows that the encoder's own rule has sensitivity 0.92–0.94 but specificity only 0.46–0.52. The clinical accuracy ceiling is therefore set by perception, not by the interface; the adapter tracks the reference (sensitivity 0.89–0.98) while default text misses more truly abnormal windows (0.67–0.71 at $N \geq 10$).

8. Auditability

- **Adapter tokens lose nothing that the vector holds.** Decoders trained on DS1 and tested on DS2 recover predicted label (99.5%) and tier (95%) from single-event tokens as well as from the 32-d input.
- **Per-event traceability survives multi-event use.** For every slot tested (0, 9, 19, 49 of a 50-event window) label, tier and urgency flag decode at the level of the input vector (Figure 8). Caveat: decoders are slot-specific (a slot-0 decoder applied at slot 49 drops to 0.64–0.91), so an auditor needs a position-aware decoder.

- **What the interface cannot carry:** heart rate, RR interval, signal quality and run length are not recoverable from the vector ($R^2 \leq 0.3$). These are exactly the fields JSON adds; none of them affects the current task.
- **Pre-generation check:** an uncertainty score computed from the vector alone predicts the adapter's wrong answers (AUROC 0.89–0.97) and flags noise and powerline interference (AUROC 0.99).

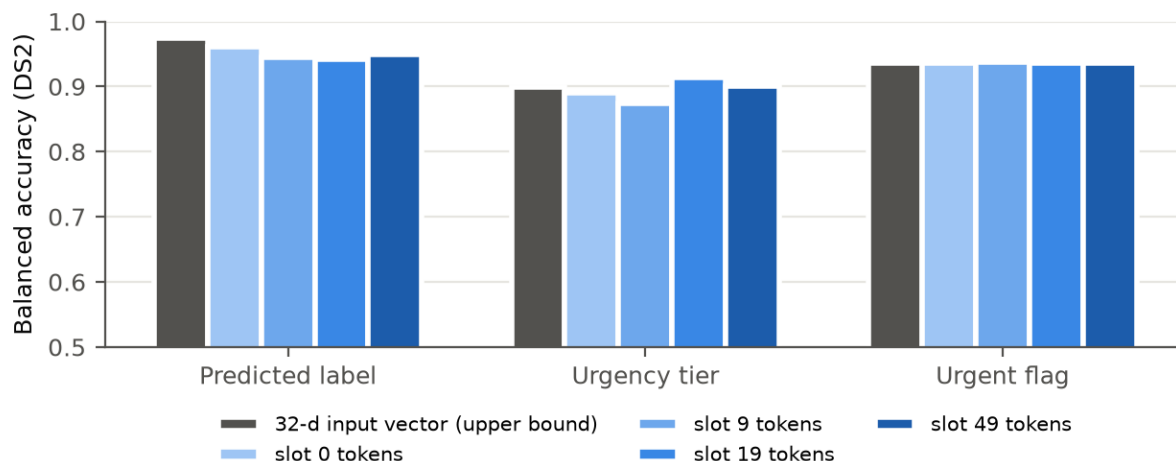


Figure 8. Balanced accuracy of decoding each event's fields from its virtual-token slot (mean of 3 seeds) versus from the encoder's 32-d vector.

9. Status of the research questions

Question	Dissertation	Now
RQ1 cost	Token and latency savings vs full JSON	No single-event saving vs fair text; 19–78% lower prefill and 4–36% faster decision at N = 10–50
RQ2 accuracy	95% (one run); 'not resolved'	Single event: worse (0.79 vs 0.97). Multi-event: non-inferior to calibrated text at N = 10–20; superior to default text at N = 10–50
RQ3 auditability	67.6% recoverability vs 100%	Cost located in the encoder; RR branch raises it to 85%; per-event decoding holds at every slot; uncertainty check available

Table 6. Research questions, dissertation vs Phase 1.

10. Limitations and threats to validity

- Many design changes (Deviations 9–17) were made after early results. Each was logged before the data it affected, but a fresh confirmatory set would remove any doubt.
- One database (MIT-BIH, 22 test patients), one language model (Gemma 4 E4B, 4-bit), one consumer GPU. Generalisation is not yet shown.
- The task is communication fidelity: the language model reproduces a fixed rule over perception outputs, which a three-line rule also solves. The contribution is about the interface, not clinical reasoning.
- The text baseline is zero-shot; the adapter is trained. A trained text baseline and a filtered text prompt (abnormal events first, normal beats summarised) are needed; the latter could narrow the efficiency advantage.
- False alarms (2.5–11.5%) and weaker single-event accuracy remain. Windowing delays alerts by up to the window length and gives one decision per window.

- The RR encoder is a single seed with 0% F-class sensitivity; perception specificity (0.46–0.52) limits clinical accuracy.
- No human evaluation; that needs ethics approval.

11. Proposed next steps and questions for the supervisor

1. **Filtered-text baseline** (abnormal events first, normal beats summarised): tests the strongest objection to the efficiency claim. About 2 h GPU.
2. **Hard-negative training (recipe r4)** with a validation set that includes low-confidence normal beats, to reduce false alarms. About 6 h GPU.
3. **Trained text baseline** (LoRA on the same training windows) for a trained-vs-trained comparison. About 5 h GPU.
4. **External validation** on another database (MIT-BIH Supraventricular Arrhythmia, INCART) and a **second language model**: the steps most likely required by a Q1 journal. Several days each.
5. Writing: corrections to the dissertation's claims stated openly as a post-viva extension; deviations summarised in one table.

Questions: (1) Which target journal should the framing aim at (clinical AI vs machine learning systems)? (2) Is external validation expected before submission, or acceptable as future work? (3) Should a small clinician rating study be planned, given the ethics lead time? (4) How should the dissertation errata be handled formally?

Appendix A. Pre-registered deviations

#	Change	Why
1–4	Correct training description; seeds; baseline family; forced-length timing	Reproduction of dissertation claims
5	Efficiency gate; generation cap 256 tokens	Claim efficiency only where CI excludes 0
6–8	Tier-balanced training; multi-event pilots; stratified pilot	Diagnose single-event limits
9	Multi-event adapter on the RR encoder	Test accuracy where efficiency exists
10	Class-balanced test windows	Test set was dominated by V beats
11	Recipe r2 (batch 1, schedule, larger validation) + early-stop minimum	r1 under-trained
12	Schema-constrained decoding (+4 corrections)	Malformed JSON, not truncation
13	N = 50 (recipe r3)	Largest efficiency gain
14	Time-to-decision latency	Latency beyond prefill
15	Exploratory robustness analyses	Patient clustering, true labels, seed vote
16 / 16a	Enlarged test set (1,195 windows); step 1 retired	Precision; claim moved to multi-event
17	Threshold calibration of both arms	False alarms; fair comparison

Table 7. Deviations 1–17 (full text in docs/analysis_plan.md).